

Code-Layout, Readability and Reusability

Date	15 July 2026
Team ID	TM-RISINGWATERS-04
Project Name	Rising Waters: AI Powered Flood Prediction System
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability Assessment:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be easily extended or reused in future deployment cycles.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code.	Yes	Black formatter utilized automatically on git commit to enforce 4-space indentations across backend and frontend.
2	Proper File Structure	Files and folders are logically organized.	Yes	Strict directory layout: separated /backend (API, logic), /frontend (React), and /ml_models (training scripts).
3	Meaningful Variable Names	Variables reflect their purpose clearly.	Yes	Used highly explicit names like <code>`water_level_sensor_input`</code> and <code>`is_alert_active`</code> instead of single-letter names.
4	Function / Method Names	Functions are descriptively named.	Yes	Verb-first snake_case syntax followed (e.g., <code>`calculate_inundation_risk()`</code> , <code>`dispatch_emergency_sms()`</code>).
5	Code Comments	Inline and block comments explain logic.	Yes	Docstrings outline complexity on all mathematical geospatial transformation and Deep Learning model functions.
6	Modular Design	Code is split into reusable functions/modules.	Yes	Created distinct utility layer for spatial calculations, external APIs, and token auth logic.
7	No Redundant Code	Duplicate or unused code is removed.	Yes	Eliminated structural duplicates with centralized helper modules; run-dead-code-check validation verified zero dead code.
8	Error Handling	Exceptions and errors are handled gracefully.	Yes	Comprehensive try-except blocks protect ML prediction routes, falling back to safe

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
				cache values on sensor drops.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	<code>raster_processor.py</code>	Python / GDAL & Rasterio	Geospatial ingest pipelines and Real-time prediction modules.	High
2	<code>useAuth.js</code> (Custom Hook)	React / JavaScript	Citizen subscription dashboard and Municipal management control boards.	High
3	<code>twilio_notifier.py</code>	Python / Twilio API	Emergency hazard triggers, test diagnostics, and monthly reports.	Medium
4	<code>LeafletMapView.jsx</code>	React / Leaflet API	Main user dashboard and administrator zoning config panel.	High
5	<code>db_connection_pool.py</code>	Python / SQLAlchemy	API Server routes, Background Celery runners, and testing suite.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5 / 5	Exceeds targets. Folder design clearly isolates logical responsibilities, facilitating onboarding and scale.
Readability	4.5 / 5	Excellent parameter naming, clear function signatures, and comprehensive module documentation.
Reusability	4.8 / 5	Core algorithms and visualization components are systematically compartmentalized for rapid deployment across sectors.
Documentation / Comments	4.5 / 5	Detailed README setup instructions, docstrings across critical routines, and autogenerated OpenAPI docs.
Overall Score	4.7 / 5	Code represents production-ready standards matching modern technical architectures.